

Q1 Modification

10 marks

What modification resolves without removing inheritance? class A { public: void show() { cout << "A"; } }; class B { public: void show() { cout << "B"; } }; class C : public A, public B {}; int main() { C obj; obj.show(); }

Your Answer

The problem occurs due to ambiguity because both classes A and B contain the same show() function. We can resolve it without removing inheritance by using the scope resolution operator (::).

```
int main()
{
    C obj;
```

Q1 Modification

10 marks

What modification resolves without removing inheritance? class A { public: void show() { cout << "A"; } }; class B { public: void show() { cout << "B"; } }; class C : public A, public B {}; int main() { C obj; obj.show(); }

Your Answer

```
obj.A::show();
}
```

Output:

A

Similarly, obj.B::show(); will call the show() function of class B.

Modification: Use A::show() or B::show() to resolve the ambiguity.

Q2 Find

10 marks

What happens if A and B own each other? struct B; struct A { shared_ptr b; }; struct B { shared_ptr<A> a; };

Your Answer

It creates a circular reference.

Because A owns B using shared_ptr, and B owns A using shared_ptr, their reference counts never become zero. Therefore, the objects are not destroyed, causing a memory leak.

Solution: Use weak_ptr for one of the references, instead of shared_ptr.

Q3 Constructor

10 marks

Reverse the construction sequence from (class A { public: A() { cout << "A "; } }; class B : public A { public: B() { cout << "B "; } }; class C : public B { public: C() { cout << "C "; } }; int main() { C obj; }) and determine the destruction order.

Your Answer

Construction order is $A \rightarrow B \rightarrow C$ because the base class constructor executes first.
 So the output is:
 ABC
 Since the question asks to reverse the construction sequence, the destruction order is the exact reverse:
 $C \rightarrow B \rightarrow A$

Q4 Virtual

10 marks

What changes if virtual is removed from Base::show() in [class Base { public: virtual void show() { cout << "Base"; } }; class Derived : public Base { public: void show() override { cout << "Derived"; } }; int main() { Base* p = new Derived(); p->show(); }]

Your Answer

If show() is declared as virtual in the base class, the derived class version is called at runtime through a base-class pointer/reference. This provides runtime polymorphism.

```
class Base {
public:
    virtual void show() {
        cout << "Base";
    }
};
```

Q4 Virtual

10 marks

What changes if virtual is removed from Base::show() in [class Base { public: virtual void show() { cout << "Base"; } }; class Derived : public Base { public: void show() override { cout << "Derived"; } }; int main() { Base* p = new Derived(); p->show(); }]

Your Answer

```
public:
    void show() override {
        cout << "Derived";
    }
};
```

So, change void show() in the base class to virtual void show().

Q5 Program

10 marks

How many temporary objects can be involved in this program (class Test { public: Test() { cout << "C "; } Test(const Test&) { cout << "Copy "; } ~Test() { cout << "D "; } Test operator+ (const Test&) { return Test(); } }; int main() { Test a, b; Test c = a + b; }).

Your Answer

2 temporary objects can be involved.
 Test() inside operator+() creates one temporary object.
 Returning it can create another temporary object through the copy constructor.
 Modern C++ may use copy elision, reducing the actual number.
 Final: 2 temporary objects (without copy elision).

Q6 Execute

10 marks

Which destructors execute? class A { public: ~A() { cout << "A "; } }; class B { public: ~B() { cout << "B "; } }; int main() { try { A a; B b; throw 10; } catch (...) { cout << "Exception"; } }

Your Answer

B A Exception

Because during exception handling, destructors execute in reverse order of object creation:

B destructor → prints B

A destructor → prints A

catch executes → prints Exception

Q7 Compile

10 marks

Will this compile? If not, identify the exact reason. `class A { public: A(int x) {} }; int main() { A obj; }`

Your Answer

No, it will not compile.

Reason: Class A has only a parameterized constructor `A(int x)`. There is no default constructor, but `A obj;` tries to call the default constructor.

Answer: No. `A obj;` is invalid because no default constructor `A()` exists.

Q8 Output

10 marks

What is the output? `class A { public: A() { cout << "A "; } A(const A&) { cout << "C "; } A& operator=(const A&) { cout << "E "; return *this; } }; int main() { A a; A b = a; A c; c = a; }`

Your Answer

Answer: ACAEA

A a; → A

A b = a; → copy constructor → C

A c; → A

c = a; → assignment operator → E

Final output: ACAE

Q9 Code

10 marks

What happens when this code executes? `class Employee { int* salary; public: Employee(int s) { salary = new int(s); } ~Employee() { delete salary; } }; int main() { Employee e1(50000); Employee e2 = e1; }`

Your Answer

The code has a shallow copy problem.
e1 and e2 point to the same dynamically allocated salary because the default copy constructor copies only the pointer.
When both objects are destroyed, delete salary is called twice, causing double deletion / undefined behavior.
Answer: e2 = e1 creates a shallow copy, so both objects share the same salary pointer.

Q10 Find

10 marks

What happens here? `class A { public: A() {} A(const A&) = delete; }; int main() { A a; A b = a; }`

Your Answer

It will not compile.

Reason: The copy constructor is written incorrectly:

`A(const A&) = delete;`

This explicitly deletes the copy constructor, so `A b = a;` is not allowed.

Answer: Compilation error because the copy constructor is deleted.